R Programming (ITA04) - CO1 Assessment Tool 1: Constraint Based Problem

Name: Sure Jaswanth Venkata Rahul

REG NO192425048

1. Student Mark Analysis using Vectors

Aim: Store the marks of 10 students in a vector and analyse them.

```
marks <- c(78, 45, 92, 67, 38, 85, 56, 71, 49, 88)
print(marks)
total   <- sum(marks)
average <- mean(marks)
print(total)
print(average)
print(max(marks))
print(min(marks))
above50 <- marks > 50
print(above50)
print(marks[above50])
print(which.max(marks))
```

Output:

```
[1] 78 45 92 67 38 85 56 71 49 88
[1] 669
[1] 66.9
[1] 92
[1] 38
 [1]  TRUE FALSE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE FALSE  TRUE
[1] 78 92 67 85 56 71 88
[1] 3
```

2. Student Information using Different Data Types

Aim: Store student details using suitable R data types and display each variable, value and type.

```
student_name <- 'Arun Kumar'
register_no  <- '212401015'
age          <- 20L
gender       <- factor('Male', levels = c('Male', 'Female', 'Other'))
marks        <- 82.5
pass_status  <- marks >= 40

info <- list(student_name = student_name, register_no = register_no,
             age = age, gender = gender, marks = marks,
             pass_status = pass_status)
```

```
writeLines(sprintf('%-12s | %-12s | %-10s | %-10s',
                   'Variable', 'Value', 'Class', 'Type'))
for (v in names(info)) {
  writeLines(sprintf('%-12s | %-12s | %-10s | %-10s',
                     v, format(info[[v]]), class(info[[v]]), typeof(info[[v]])))
}
```

Output:

```
Variable     | Value        | Class      | Type
student_name | Arun Kumar   | character  | character
register_no  | 212401015    | character  | character
age          | 20           | integer    | integer
gender       | Male         | factor     | integer
marks        | 82.5         | numeric    | double
pass_status  | TRUE         | logical    | logical
```

3. Matrix Operations in R

Aim: Perform matrix addition, subtraction, multiplication, transpose and element extraction on marks in two subjects.

```
A <- matrix(c(78, 85, 90,
              66, 72, 88,
              59, 94, 81), nrow = 3, byrow = TRUE)   # Subject 1
B <- matrix(c(70, 80, 65,
              88, 76, 92,
              73, 69, 84), nrow = 3, byrow = TRUE)   # Subject 2

print(A)
print(B)

print(A + B)        # element-wise addition
print(A - B)        # element-wise subtraction
print(A %*% B)      # matrix multiplication (row x column)
print(t(A))         # transpose of A

# Extraction of selected elements
print(A[2, 3])      # element in row 2, column 3
print(A[1, ])       # entire first row
print(B[, 2])       # entire second column
print(A[2:3, 1:2])  # sub-matrix
```

Output:

```
> A + B
     [,1] [,2] [,3]
[1,]  148  165  155
[2,]  154  148  180
[3,]  132  163  165
```

```
> A - B
     [,1] [,2] [,3]
[1,]    8    5   25
[2,]  -22   -4   -4
[3,]  -14   25   -3

> A %*% B
      [,1]  [,2]  [,3]
[1,] 19510 18910 20450
[2,] 17380 16824 18306
[3,] 18315 17453 19287

> t(A)
     [,1] [,2] [,3]
[1,]   78   66   59
[2,]   85   72   94
[3,]   90   88   81

> A[2, 3]
[1] 88
> A[1, ]
[1] 78 85 90
> B[, 2]
[1] 80 76 69
> A[2:3, 1:2]
     [,1] [,2]
[1,]   66   72
[2,]   59   94
```

4. Employee Salary Analysis using Lists

Aim: Store employee details in a list and calculate gross and net salary.\

```
employee <- list(
  name        = 'Priya Sharma',
  emp_id      = 'E1023',
  department  = 'Finance',
  basic       = 45000,
  allowances  = 12000,
  deductions  = 5500
)

# Calculations using list components
gross_salary <- employee$basic + employee$allowances
net_salary   <- gross_salary - employee$deductions

# Display individual components
writeLines(paste('Name        :', employee$name))
writeLines(paste('Employee ID :', employee[['emp_id']]))
writeLines(paste('Department  :', employee$department))
writeLines(paste('Basic Salary:', employee$basic))
writeLines(paste('Allowances  :', employee$allowances))
```

```
writeLines(paste('Deductions  :', employee$deductions))
writeLines(paste('Gross Salary:', gross_salary))
writeLines(paste('Net Salary  :', net_salary))
employee$gross <- gross_salary
employee$net   <- net_salary
str(employee)
```

Output:

```
Name        : Priya Sharma
Employee ID : E1023
Department  : Finance
Basic Salary: 45000
Allowances  : 12000
Deductions  : 5500
Gross Salary: 57000
Net Salary  : 51500
List of 8
 $ name       : chr "Priya Sharma"
 $ emp_id     : chr "E1023"
 $ department : chr "Finance"
 $ basic      : num 45000
 $ allowances : num 12000
 $ deductions : num 5500
 $ gross      : num 57000
 $ net        : num 51500
```

5. Product Sales using Arrays

Aim: Store sales of 3 products, in 2 branches, over 2 months in a 3-dimensional array.

```
sales <- array(
  c(120, 150,  90,   # Jan - Chennai   (P1, P2, P3)
    110, 130,  80,   # Jan - Bengaluru
    140, 160, 100,   # Feb - Chennai
    125, 145,  95),  # Feb - Bengaluru
  dim = c(3, 2, 2),
  dimnames = list(
    Product = c('P1', 'P2', 'P3'),
    Branch  = c('Chennai', 'Bengaluru'),
    Month   = c('Jan', 'Feb')
  )
)

print(sales)

# Access individual values
print(sales['P2', 'Chennai', 'Feb'])   # by names
print(sales[3, 2, 1])                  # by index
print(sales[, , 'Jan'])                # all January sales

# Totals
print(sum(sales))                      # total sales
```

```
print(apply(sales, 1, sum))          # total per product
print(apply(sales, 2, sum))          # total per branch
print(apply(sales, 3, sum))          # total per month
```

Output (key results):

```
> sales['P2', 'Chennai', 'Feb']
[1] 160
> sales[3, 2, 1]
[1] 80
> sum(sales)
[1] 1445
> apply(sales, 1, sum)
  P1  P2  P3
495 585 365
> apply(sales, 2, sum)
  Chennai Bengaluru
      760       685
> apply(sales, 3, sum)
 Jan  Feb
 680  765
```

6. Student Grade Analysis using Factors

Aim: Represent the grades of 10 students as a factor and analyse them.

```
grades <- factor(c('A', 'B', 'C', 'B', 'A', 'D', 'C', 'B', 'A', 'C'),
                 levels  = c('A', 'B', 'C', 'D'),
                 ordered = TRUE)

print(grades)               # the factor
print(levels(grades))       # factor levels
print(nlevels(grades))      # number of levels
print(table(grades))        # number of students in each grade
print(grades[3])            # access individual element
print(grades[c(1, 6, 10)])  # access several elements
print(as.integer(grades))   # internal integer codes
print(grades[1] < grades[6])# ordered comparison: A < D ?
```

Output:

```
[1] A B C B A D C B A C
Levels: A < B < C < D

> levels(grades)
[1] "A" "B" "C" "D"
> nlevels(grades)
[1] 4
> table(grades)
grades
A B C D
3 3 3 1
> grades[3]
```

```
[1] C
Levels: A < B < C < D
> grades[c(1, 6, 10)]
[1] A D C
Levels: A < B < C < D
> as.integer(grades)
 [1] 1 2 3 2 1 4 3 2 1 3
> grades[1] < grades[6]
[1] TRUE
```

7. Student Data Frame Problem

Aim: Create a data frame of five students and analyse the marks.

```
students <- data.frame(
  StudentID  = c('S101', 'S102', 'S103', 'S104', 'S105'),
  Name       = c('Arun', 'Divya', 'Karthik', 'Meena', 'Suresh'),
  Age        = c(19, 20, 19, 21, 20),
  Department = c('CSE', 'IT', 'ECE', 'CSE', 'IT'),
  Gender     = c('Male', 'Female', 'Male', 'Female', 'Male'),
  Marks      = c(85, 72, 91, 64, 78),
  stringsAsFactors = FALSE
)

print(students)              # display contents
str(students)               # structure
print(students$Name)
print(students[['Marks']])
print(students[, c('Name', 'Department')])
print(students[2, ])         # second student
print(students[c(1, 3), ])    # rows 1 and 3
print(students[students$Marks > 75, ])   # rows by condition
print(mean(students$Marks))
print(max(students$Marks))
print(min(students$Marks))
```

Output:

```
  StudentID    Name Age Department Gender Marks
1      S101    Arun  19        CSE   Male    85
2      S102   Divya  20         IT Female    72
3      S103 Karthik  19        ECE   Male    91
4      S104   Meena  21        CSE Female    64
5      S105  Suresh  20         IT   Male    78

> mean(students$Marks)
[1] 78
> max(students$Marks)
[1] 91
> min(students$Marks)
[1] 64
```

8. Logical Vector-Based Student Selection

Aim: Classify 15 students using logical-vector operations.

```
marks <- c(82, 45, 67, 91, 38, 55, 76, 49, 63, 88, 72, 34, 59, 95, 50)

# Logical vectors
above75  <- marks > 75
between  <- marks >= 50 & marks <= 75
below50  <- marks < 50
passed   <- marks >= 50

writeLines(paste('a) Above 75        :', paste(marks[above75], collapse = ' ')))
writeLines(paste('b) Between 50 & 75 :', paste(marks[between], collapse = ' ')))
writeLines(paste('c) Below 50        :', paste(marks[below50], collapse = ' ')))
writeLines(paste('d) Passed          :', paste(marks[passed], collapse = ' ')))

# Positions (student numbers) and counts
print(which(above75))
writeLines(paste('Number of students passed:', sum(passed)))
```

Output:

```
a) Above 75        : 82 91 76 88 95
b) Between 50 & 75 : 67 55 63 72 59 50
c) Below 50        : 45 38 49 34
d) Passed          : 82 67 91 55 76 63 88 72 59 95 50
[1]  1  4  7 10 14
Number of students passed: 11
```

9. R Data Structure Problem

Aim: Represent student information with vectors, lists, matrices, factors and data frames.

```
marks_vec <- c(85, 72, 91, 64, 78)
print(marks_vec)
print(marks_vec[3])              # access by position
student_list <- list(name = 'Arun', reg_no = '212401015',
                     age = 19L, marks = c(85, 78, 90), passed = TRUE)
print(student_list$name)         # access by name
print(student_list[['marks']])   # access by name with [[ ]]
marks_mat <- matrix(c(85, 78, 90,
                      72, 68, 75,
                      91, 88, 84), nrow = 3, byrow = TRUE,
                    dimnames = list(c('S1', 'S2', 'S3'),
                                    c('Maths', 'Science', 'English')))
print(marks_mat)
print(marks_mat['S2', 'Science'])  # access by names

dept <- factor(c('CSE', 'IT', 'ECE', 'CSE', 'IT'))
print(levels(dept))
print(table(dept))
df <- data.frame(Name = c('Arun', 'Divya', 'Karthik', 'Meena', 'Suresh'),
                 Dept = dept, Marks = marks_vec)
```

```
print(df)
print(df$Marks)              # a column
print(df[2, ])               # a row
```

Output (selected):

```
[1] 85 72 91 64 78
[1] 91

> marks_mat
   Maths Science English
S1    85      78      90
S2    72      68      75
S3    91      88      84

> table(dept)
dept
CSE ECE  IT
  2   1   2

> df
     Name Dept Marks
1    Arun  CSE    85
2   Divya   IT    72
3 Karthik  ECE    91
4   Meena  CSE    64
5  Suresh   IT    78
```

10. Student Result-Processing Application

Aim: Process the results of five students using vectors, a result matrix, factors (grades) and a data frame.

```
name    <- c('Arun', 'Divya', 'Karthik', 'Meena', 'Suresh')
maths   <- c(85, 72, 91, 64, 78)
science <- c(78, 68, 88, 70, 82)
english <- c(90, 75, 84, 60, 80)
total   <- maths + science + english
average <- round(total / 3, 2)
result_matrix <- cbind(Maths = maths, Science = science, English = english,
                       Total = total, Average = average)
rownames(result_matrix) <- paste0('S', 1:5)
print(result_matrix)
grade <- cut(average,
             breaks = c(0, 50, 65, 80, Inf),
             labels = c('D', 'C', 'B', 'A'),
             right  = FALSE)         # A >= 80, B >= 65, C >= 50, D < 50

status <- ifelse(average >= 50, 'Pass', 'Fail')
final_result <- data.frame(Name = name, Maths = maths, Science = science,
                           English = english, Total = total,
                           Average = average, Grade = grade, Status = status)
print(final_result)
for (i in seq_len(nrow(final_result))) {
```

```
  writeLines(paste0(final_result$Name[i], ' scored ', final_result$Total[i],
                    ' marks, Grade ', final_result$Grade[i],
                    ' (', final_result$Status[i], ')'))
}
print(table(final_result$Grade))
writeLines(paste('Topper:', final_result$Name[which.max(final_result$Total)]))
```

Output:

```
  Maths Science English Total Average
S1    85      78      90   253   84.33
S2    72      68      75   215   71.67
S3    91      88      84   263   87.67
S4    64      70      60   194   64.67
S5    78      82      80   240   80.00

     Name Maths Science English Total Average Grade Status
1    Arun    85      78      90   253   84.33     A   Pass
2   Divya    72      68      75   215   71.67     B   Pass
3 Karthik    91      88      84   263   87.67     A   Pass
4   Meena    64      70      60   194   64.67     C   Pass
5  Suresh    78      82      80   240   80.00     A   Pass

Arun scored 253 marks, Grade A (Pass)
Divya scored 215 marks, Grade B (Pass)
Karthik scored 263 marks, Grade A (Pass)
Meena scored 194 marks, Grade C (Pass)
Suresh scored 240 marks, Grade A (Pass)

A B C D
3 1 1 0

Topper: Karthik
```

Code of Conduct:

I Sure Jaswanth Venkata Rahul (192425048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S Jaswanth

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	**20%**	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	**30%**	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	**20%**	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	**10%**	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	**20%**	Handles all cases; optimized Approach	Handles most cases reasonable efficiency	Handles basic cases only	Incomplete or inefficient